leanprover-community.github.io

Library Style Guidelines

28–35 minutes

In addition to the naming conventions, files in the Lean library generally adhere to the following guidelines and conventions. Having a uniform style makes it easier to browse the library and read the contents, but these are meant to be guidelines rather than rigid rules.

Variable conventions #

- u, v, w, ... for universes
- α, β, γ, ... for generic types
- a, b, c, ... for propositions
- x, y, z, ... for elements of a generic type
- h, h_1, ... for assumptions
- p, q, r, ... for predicates and relations
- s, t, ... for lists
- s, t, ... for sets
- m, n, k, ... for natural numbers
- i, j, k, ... for integers

Types with a mathematical content are expressed with the usual mathematical notation, often with an upper case letter (G for a group, R for a ring, K or 𝕜 for a field, E for a vector space, ...). This convention is not followed in older files, where greek letters are used for all types. Pull requests renaming type variables in these files are welcome.

Unicode usage #

Use special Unicode characters, especially mathematical symbols where they help develop good notation and improve readability. Avoid the following:

- characters which change text direction
- invisible characters (except spaces and newlines)
- characters which modify other characters

Mathlib has a linter which checks all characters against an allow-list, to which new characters may be added as required.

Line length #

Lines should not be longer than 100 characters. This makes files easier to read, especially on a small screen or in a small window. If you are editing with VS Code, there is a visual marker which will indicate a 100 character limit.

Header and imports

The file header should contain copyright information, a list of all the authors who have made significant contributions to the file, and a description of the contents. Put the `module` keyword on its own line right after the header, skip a line, then group all `public import`s together, skip another line, then group all `import`s together, Try to keep the imports alphabetical within each block of imports.

```
/-
Copyright (c) 2024 Joe Cool. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Joe Cool
-/
module

public import Mathlib.Logic.Defs

import Mathlib.Algebra.Group.Defs
import Mathlib.Data.Nat.Basic
```

(Tip: If you're editing mathlib in VS Code, you can write `copy` and then press TAB to generate a skeleton of the copyright header.)

Regarding the list of authors: use `Authors` even when there is only a single author. Don't end the line with a period, and use commas (`, `) to separate all author names (so don't use `and` between the penultimate and final author.) We don't have strict rules on what contributions qualify for inclusion there. The general idea is that the people listed there should be the ones we would reach out to if we had questions about the design or development of the Lean code.

Note that there are less common combinations of keywords such as `public meta import` or `import all`. Given their rarity, we do not yet prescribe their relative location within the import statements.

Module docstrings

After the copyright header and the imports, please add a module docstring (delimited with `/-!` and `-/`) containing

- a title of the file,
- a summary of the contents (the main definitions and theorems, proof techniques, etc...)
- notation that has been used in the file (if any)
- references to the literature (if any)

In total, the module docstring should look something like this:

```
/-!
# Foos and bars

In this file we introduce `foo` and `bar`,
two main concepts in the theory of xyzzyology.

## Main results

- `exists_foo`: the main existence theorem of `foo`s.
- `bar_of_foo_of_baz`: a construction of a `bar`, given a `foo` and a `baz`.
  If this doc-string is longer than one line, subsequent lines should be
```

```
indented by two spaces
  (as required by markdown syntax).
- `bar_eq`    : the main classification theorem of `bar`s.

## Notation

 - `|_|` : The barrification operator, see `bar_of_foo`.

## References

See [Thales600BC] for the original account on Xyzzyology.
-/
```

New bibliography entries should be added to `docs/references.bib`.

See our documentation requirements for more suggestions and examples.

Structuring definitions and theorems #

All declarations (e.g., `def`, `lemma`, `theorem`, `class`, `structure`, `inductive`, `instance`, etc.) and commands (e.g., `variable`, open, `section`, `namespace`, `notation`, etc.) are considered top-level and these words should appear flush-left in the document. In particular, opening a namespace or section does not result in indenting the contents of that namespace or section. (Note: within VS Code, hovering over any declaration such as `def Foo ...` will show the fully qualified name, like `MyNamespace Foo` if Foo is declared while the namespace `MyNamespace` is open.)

These guidelines hold for declarations starting with `def`, `lemma` and `theorem`. For "theorem statement", also read "type of a definition" and for "proof" also read "definition body".

Use spaces on both sides of ":", ":=" or infix operators. Put them before a line break rather than at the beginning of the next line.

In what follows, "indent" without an explicit indication of the amount means "indent by 2 additional spaces".

After stating the theorem, we indent the lines in the subsequent proof by 2 spaces.

```
open Nat
theorem nat_case {P : Nat → Prop} (n : Nat) (H1 : P 0) (H2 : ∀ m, P (succ m)) :
P n :=
  Nat.recOn n H1 (fun m IH ↦ H2 m)
```

If the theorem statement requires multiple lines, indent the subsequent lines by 4 spaces. The proof is still indented only 2 spaces (*not* 6 = 4 + 2). When providing a proof in tactic mode, the by is placed on the line *prior* to the first tactic; however, by should not be placed on a line by itself. In practice this means you will often see `:= by` at the end of a theorem statement.

```
import Mathlib.Data.Nat.Basic

theorem le_induction {P : Nat → Prop} {m}
    (h0 : P m) (h1 : ∀ n, m ≤ n → P n → P (n + 1)) :
    ∀ n, m ≤ n → P n := by
  apply Nat.le.rec
  · exact h0
  · exact h1 _

def decreasingInduction {P : ℕ → Sort*} (h : ∀ n, P (n + 1) → P n) {m n : ℕ}
(mn : m ≤ n)
```

```
    (hP : P n) : P m :=
  Nat.leRecOn mn (fun {k} ih hsk => ih <| h k hsk) (fun h => h) hP
```

When a proof term takes multiple arguments, it is sometimes clearer, and often necessary, to put some of the arguments on subsequent lines. In that case, indent each argument. This rule, i.e., indent an additional two spaces, applies more generally whenever a term spans multiple lines.

```
open Nat
axiom zero_or_succ (n : Nat) : n = zero ∨ n = succ (pred n)
theorem nat_discriminate {B : Prop} {n : Nat} (H1: n = 0 → B) (H2 : ∀ m, n =
succ m → B) : B :=
  Or.elim (zero_or_succ n)
    (fun H3 : n = zero ↦ H1 H3)
    (fun H3 : n = succ (pred n) ↦ H2 (pred n) H3)
```

Don't orphan parentheses; keep them with their arguments.

Here is a longer example.

```
import Mathlib.Init.Data.List.Lemmas

open List
variable {T : Type}

theorem mem_split {x : T} {l : List T} : x ∈ l → ∃ s t : List T, l = s ++ (x ::
t) :=
  List.recOn l
    (fun H : x ∈ [] ↦ False.elim ((mem_nil_iff _).mp H))
    (fun y l ↦
      fun IH : x ∈ l → ∃ s t : List T, l = s ++ (x :: t) ↦
      fun H : x ∈ y :: l ↦
      Or.elim (eq_or_mem_of_mem_cons H)
        (fun H1 : x = y ↦
          Exists.intro [] (Exists.intro l (by rw [H1]; rfl)))
        (fun H1 : x ∈ l ↦
          let ⟨s, (H2 : ∃ t : List T, l = s ++ (x :: t))⟩ := IH H1
          let ⟨t, (H3 : l = s ++ (x :: t))⟩ := H2
          have H4 : y  ::  l = (y :: s) ++ (x :: t) := by rw [H3]; rfl
          Exists.intro (y :: s) (Exists.intro t H4)))
```

The type of all arguments of a declaration should be given explicitly, even if Lean can figure out this type information by itself. This makes it easier to understand the definition when seeing it on a webpage like GitHub. For the same reason, the return type of all declarations should also be given (Lean enforces this only for theorems). So you should follow the style of `GoodStatement` in this example:

```
def BadStatement (n) := ∃ k, n + k = 3
def GoodStatement (n : ℕ) : Prop := ∃ k : ℕ, n + k = 3
```

A short declaration can be written on a single line:

```
open Nat
theorem succ_pos : ∀ n : Nat, 0 < succ n := zero_lt_succ

def square (x : Nat) : Nat := x * x
```

A have can be put on a single line when the justification is short.

```
example (n k : Nat) (h : n < k) : ... :=
  have h1 : n ≠ k := ne_of_lt h
  ...
```

When the justification is too long, you should put it on the next line, indented by an additional two spaces.

```
example (n k : Nat) (h : n < k) : ... :=
  have h1 : n ≠ k :=
    ne_of_lt h
  ...
```

When the justification of the have uses tactic mode, the by should be placed on the same line, regardless of whether the justification spans multiple lines.

```
example (n k : Nat) (h : n < k) : ... :=
  have h1 : n ≠ k := by apply ne_of_lt; exact h
  ...

example (n k : Nat) (h : n < k) : ... :=
  have h1 : n ≠ k := by
    apply ne_of_lt
    exact h
  ...
```

When the arguments themselves are long enough to require line breaks, use an additional indent for every line after the first, as in the following example:

```
import Mathlib.Data.Nat.Basic

theorem Nat.add_right_inj {n m k : Nat} : n + m = n + k → m = k :=
  Nat.recOn n
    (fun H : 0 + m = 0 + k ↦ calc
      m = 0 + m := Eq.symm (zero_add m)
      _ = 0 + k := H
      _ = k     := zero_add _)
    (fun (n : Nat) (IH : n + m = n + k → m = k) (H : succ n + m = succ n + k)
↦
      have H2 : succ (n + m) = succ (n + k) := calc
        succ (n + m) = succ n + m   := Eq.symm (succ_add n m)
        _            = succ n + k   := H
        _            = succ (n + k) := succ_add n k
      have H3 : n + m = n + k := succ.inj H2
      IH H3)
```

In a class or structure definition, fields are indented 2 spaces, and moreover each field should have a docstring, as in:

```
structure PrincipalSeg {α β : Type*} (r : α → α → Prop) (s : β → β → Prop)
extends r ↪r s where
  /-- The supremum of the principal segment -/
  top : β
  /-- The image of the order embedding is the set of elements `b` such that `s
b top` -/
  down' : ∀ b, s b top ↔ ∃ a, toRelEmbedding a = b
```

```
class Module (R : Type u) (M : Type v) [Semiring R] [AddCommMonoid M] extends
    DistribMulAction R M where
  /-- Scalar multiplication distributes over addition from the right. -/
  protected add_smul : ∀ (r s : R) (x : M), (r + s) • x = r • x + s • x
  /-- Scalar multiplication by zero gives zero. -/
  protected zero_smul : ∀ x : M, (0 : R) • x = 0
```

Subsequent declarations should be separated by a single line break. Exception is made for groups of similar one-line declarations.

```
theorem foo : True :=
  sorry

theorem bar : False :=
  sorry

@[simp] theorem one_lt_two : 1 < 2 := sorry
@[simp] theorem two_lt_three : 2 < 3 := sorry
```

When using a constructor taking several arguments in a definition, arguments line up, as in:

```
theorem Ordinal.sub_eq_zero_iff_le {a b : Ordinal} : a - b = 0 ↔ a ≤ b :=
  ⟨fun h => by simpa only [h, add_zero] using le_add_sub a b,
   fun h => by rwa [← Ordinal.le_zero, sub_le, add_zero]⟩
```

Instances #

When providing terms of structures or instances of classes, the `where` syntax should be used to avoid the need for enclosing braces, as in:

```
instance instOrderBot : OrderBot ℕ where
  bot := 0
  bot_le := Nat.zero_le
```

If there is already an instance `instBot`, then one can write

```
instance instOrderBot : OrderBot ℕ where
  __ := instBot
  bot_le := Nat.zero_le
```

Hypotheses Left of Colon #

Generally, having arguments to the left of the colon is preferred over having arguments in universal quantifiers or implications, if the proof starts by introducing these variables. For instance:

```
example (n : ℝ) (h : 1 < n) : 0 < n := by linarith
```

is preferred over

```
example (n : ℝ) : 1 < n → 0 < n := fun h ↦ by linarith
```

and

```
example (n : ℕ) : 0 ≤ n := Nat.zero_le n
```

is preferred over

```
example : ∀ (n : ℕ), 0 ≤ n := Nat.zero_le
```

Note that pattern-matching does not count as the proof starting by introducing variables. For example, the following is a valid use case of having a hypothesis right of the column:

```
lemma zero_le : ∀ n : ℕ, 0 ≤ n
  | 0 => le_rfl
  | n + 1 => add_nonneg (zero_le n) zero_le_one
```

Binders #

Use a space after binders. Also, the binder type should generally be written explicitly, even if Lean doesn't need this information.

```
example : ∀ α : Type, ∀ x : α, ∃ y : α, y = x :=
  fun (α : Type) (x : α) ↦ Exists.intro x rfl
```

Anonymous functions #

Lean has several nice syntax options for declaring anonymous functions. For very simple functions, one can use the centered dot as the function argument, as in `(· ^ 2)` to represent the squaring function. However, sometimes it is necessary to refer to the arguments by name (e.g., if they appear in multiple places in the function body). The Lean default for this is `fun x => x * x`, but the `↦` arrow (inserted with `\mapsto`) is also valid. In mathlib the pretty printer displays `↦`, and we slightly prefer this in the source as well. The lambda notation `λ x ↦ x * x`, while syntactically valid, is disallowed in mathlib in favor of the `fun` keyword.

Calculations #

There is some flexibility in how you write calculational proofs, although there are some rules enforced by the syntax requirements of `calc` itself. However, there are some general guidelines.

As with by, the `calc` keyword should be placed on the line *prior* to the start of the calculation, with the calculation indented. Whichever relations are involved (e.g., = or ≤) should be aligned from one line to the next. The underscores _ used as placeholders for terms indicating the continuation of the calculation should be left-justified.

As for the justifications, it is not necessary to align the := symbols, but it can be nice if the expressions are short enough. The terms on either side of the first relation can either go on one line or separate lines, which may be decided by the size of the expressions.

An example of adequate style which can more easily accommodate longer expressions is:

```
import Init.Data.List.Basic

open List

theorem reverse_reverse : ∀ (l : List α), reverse (reverse l) = l
  | []     => rfl
  | a :: l => calc
    reverse (reverse (a :: l))
      = reverse (reverse l ++ [a]) := by rw [reverse_cons]
    _ = reverse [a] ++ reverse (reverse l) := reverse_append _ _
    _ = reverse [a] ++ l := by rw [reverse_reverse l]
    _ = a :: l := rfl
```

The following style has the substantial advantage of having all lines be interchangeable, which is particularly useful when editing the proof in eg VSCode:

```
import Init.Data.List.Basic

open List
```

```
theorem reverse_reverse : ∀ (l : List α), reverse (reverse l) = l
  | []     => rfl
  | a :: l => calc
      reverse (reverse (a :: l))
    _ = reverse (reverse l ++ [a]) := by rw [reverse_cons]
    _ = reverse [a] ++ reverse (reverse l) := reverse_append _ _
    _ = reverse [a] ++ l := by rw [reverse_reverse l]
    _ = a :: l := rfl
```

If the expressions and proofs are relatively short, the following style is also an option:

```
import Init.Data.List.Basic

open List

theorem reverse_reverse : ∀ (l : List α), reverse (reverse l) = l
  | []     => rfl
  | a :: l => calc
    reverse (reverse (a :: l)) = reverse (reverse l ++ [a])         := by rw
[reverse_cons]
    _                          = reverse [a] ++ reverse (reverse l) :=
reverse_append _ _
    _                          = reverse [a] ++ l                   := by rw
[reverse_reverse l]
    _                          = a :: l                             := rfl
```

Tactic mode #

As we have already mentioned, when opening a tactic block, by is placed at the end of the line *preceding* the start of the tactic block, but not on its own line. Everything within the tactic block is indented, as in:

```
theorem continuous_uncurry_of_discreteTopology [DiscreteTopology α] {f : α → β
→ γ}
    (hf : ∀ a, Continuous (f a)) : Continuous (uncurry f) := by
  apply continuous_iff_continuousAt.2
  rintro ⟨a, x⟩
  change map _ _ ≤ _
  rw [nhds_prod_eq, nhds_discrete, Filter.map_pure_prod]
  exact (hf a).continuousAt
```

One can mix term mode and tactic mode, as in:

```
theorem Units.isUnit_units_mul {M : Type*} [Monoid M] (u : Mˣ) (a : M) :
    IsUnit (↑u * a) ↔ IsUnit a :=
  Iff.intro
    (fun ⟨v, hv⟩ => by
      have : IsUnit (↑u⁻¹ * (↑u * a)) := by exists u⁻¹ * v; rw [← hv,
Units.val_mul]
      rwa [← mul_assoc, Units.inv_mul, one_mul] at this)
    u.isUnit.mul
```

When new goals arise as side conditions or steps, they are indented and preceded by a focusing dot · (inserted as \.); the dot is not indented.

```
import Mathlib.Algebra.Group.Basic
```

```
theorem exists_npow_eq_one_of_zpow_eq_one' [Group G] {n : ℤ} (hn : n ≠ 0) {x :
G} (h : x ^ n = 1) :
    ∃ n : ℕ, 0 < n ∧ x ^ n = 1 := by
  cases n
  · simp only [Int.ofNat_eq_coe] at h
    rw [zpow_ofNat] at h
    refine ⟨_, Nat.pos_of_ne_zero fun n0 ↦ hn ?_, h⟩
    rw [n0]
    rfl
  · rw [zpow_negSucc, inv_eq_one] at h
    refine ⟨_ + 1, Nat.succ_pos _, h⟩
```

Certain tactics, such as `refine`, can create *named* subgoals which can be proven in whichever order is desired using `case`. This feature is also useful in aiding readability. However, it is not required to use this instead of the focusing dot (`·`).

```
example {p q : Prop} (h₁ : p → q) (h₂ : q → p) : p ↔ q := by
  refine ⟨?imp, ?converse⟩
  case converse => exact h₂
  case imp => exact h₁
```

Often `t0 <;> t1` is used to execute `t0` and then `t1` on all new goals. Either write the tactics in one line, or indent the following tactic.

```
  cases x <;>
    simp [a, b, c, d]
```

For single line tactic proofs (or short tactic proofs embedded in a term), it is acceptable to use by `tac1; tac2; tac3` with semicolons instead of a new line with indentation.

In general, you should put a single tactic invocation per line, unless you are closing a goal with a proof that fits entirely on a single line. Short sequences of tactics that correspond to a single mathematical idea can also be put on a single line, separated by semicolons as in `cases bla; clear h` or `induction n; simp` or `rw [foo]; simp_rw [bar]`, but even in these scenarios, newlines are preferred.

```
example : ... := by
  by_cases h : x = 0
  · rw [h]; exact hzero ha
  · rw [h]
    have h' : ... := H ha
    simp_rw [h', hb]
    ...
```

Very short goals can be closed right away using `swap` or `pick_goal` if needed, to avoid additional indentation in the rest of the proof.

```
example : ... := by
  rw [h]
  swap; exact h'
  ...
```

We generally use a blank line to separate theorems and definitions, but this can be omitted, for example, to group together a number of short definitions, or to group together a definition and notation.

Squeezing simp calls #

Unless performance is particularly poor or the proof breaks otherwise, *terminal `simp` calls* (a `simp` call is terminal if it closes the current goal or is only followed by flexible tactics such as `ring`, `field_simp`, `aesop`) should not be *squeezed* (replaced by the output of `simp?`).

There are two main reasons for this:

1. A squeezed `simp` call might be several lines longer than the corresponding unsqueezed one, and therefore drown the useful information of what key lemmas were added to the unsqueezed `simp` call to close the goal in a sea of basic simp lemmas.
2. A squeezed `simp` call refers to many lemmas by name, meaning that it will break when one such lemma gets renamed. Lemma renamings happen often enough for this to matter on a maintenance level.

Profiling for performance #

When contributing to mathlib, authors should be aware of the performance impacts of their contributions. The Lean FRO maintains benchmarking infrastructure which can be accessed by commenting `!bench` on a PR.

Authors should assure that their contributions do not cause significant performance regressions. In particular, if the PR touches significant components of the language like adding new classes, instances, or `simp` lemmas, changing imports, creating new definitions, or turning `def`s into `abbrev`s, then authors should benchmark their changes proactively. Nontrivial `refactor` PRs, in particular should be benchmarked and any significant negative results must be explained during the review process.

Transparency and API design #

Central to Lean being a practically performant proof assistant is avoiding checking of definitional equality for very large terms. In the elaborator (the component of the language that converts syntax to terms), the notion of transparency is the main mechanism to avoid unfolding large definitions when unnecessary. Excluding opaque definitions, there are three levels of transparency:

- `reducible` definitions are always unfolded
- `semireducible` definitions (the default) are usually not unfolded in main tactics like `rw` and `simp`, but can be unfolded with a little effort like explicitly calling `rfl` or `erw`. Semireducible definitions are also not unfolded during the computation of keys for storing instances in the instance cache or simp lemmas in the simp cache.
- `irreducible` definitions are never unfolded unless the user explicitly requests it (e.g using the `unfold` tactic, or by using the `unseal` command).

`def` by default creates `semireducible` definitions and `abbrev` creates `reducible` (and `@[inline]`) definitions.

When designing definitions, an author should give thought to the transparency level of definitions. Consider how exposing the underlying term of your definition will affect instance search and simplification. The default for mathlib is that definitions should be `semireducible` unless there is a good reason otherwise which should be clearly articulated in the PR description. This imposes overhead on contributors who will need to declare new instances of the form

```
instance : Foo myDef := inferInstanceAs (Foo underlyingTermOfMyDef)
```

and recycle API lemmas, especially for `simp` use, like

```
@[simp] lemma myDef_bar_eq_bizz (x : X) : myDef.bar = bizz :=
    underlyingTermOfMyDef_bar_eq_bizz
```

If the API boundary is meant to be completely sealed, using a type synonym of the form

```
structure myDef where
   underlying : underlyingTerm
```

is the library convention in place of `irreducible` definitions. These structure wrappers are intended for types that are equivalent to an existing type but are clearly mathematically semantically distinct, e.g. `Option` and `WithTop`.

The kernel does not have an analogous notion of transparency so its rules for unfolding are different. There are situations where an author wants to block unfolding in the kernel as well. Mathlib provides a command `irreducible_def` for this. This should be used only when there is a documented necessity from profiling.

Use of `erw` or `rfl` after tactics like `simp` or `rw` that operate at reducible transparency is an indication that there is missing API. Consider adding the necessary lemmas to the API to avoid this.

The library has existing occurrences of definitional transparency abuse like `erw` and extra `rfl`. PRs removing these are very welcome but their PR description should clearly articulate how the removal is achieved addressing the change in the underlying terms in particular and must benchmark their changes. Please treat this an opportunity to improve the API design of the relevant components.

Whitespace and delimiters #

Lean is whitespace-sensitive, and in general we opt for a style which avoids delimiting code. For instance, when writing tactics, it is possible to write them as `tac1; tac2; tac3`, separated by `;`, in order to override the default whitespace sensitivity. However, as mentioned above, we generally try to avoid this except in a few special cases.

Similarly, sometimes parentheses can be avoided by judicious use of the `<|` operator (or its cousin `|>`). Note: while `$` is a synonym for `<|`, its use in mathlib is disallowed in favor of `<|` for consistency as well as because of the symmetry with `|>`. These operators have the effect of parenthesizing everything to the right of `<|` (note that `(` is curved the same direction as `<`) or to the left of `|>` (and `)` curves the same way as `>`).

A common example of the usage of `|>` occurs with dot notation when the term preceding the `.` is a function applied to some arguments. For instance, `((foo a).bar b).baz` can be rewritten as `foo a |>.bar b |>.baz`

A common example of the usage of `<|` is when the user provides a term which is a function applied to multiple arguments whose last argument is a proof in tactic mode, especially one that spans multiple lines. In that case, it is natural to use `<| by ...` instead of `(by ...)`, as in:

```
import Mathlib.Tactic

example {x y : ℝ} (hxy : x ≤ y) (h : ∀ ε > 0, y - ε ≤ x) : x = y :=
  le_antisymm hxy <| le_of_forall_pos_le_add <| by
    intro ε hε
    have := h ε hε
    linarith
```

When using the tactics `rw` or `simp` there should be a space after the left arrow `←`. For instance `rw [← add_comm a b]` or `simp [← and_or_left]`. (There should also be a space between the tactic name and its arguments, as in `rw [h]`.) This rule applies the do notation as well: `do return (← f) + (← g)`

Empty lines inside declarations #

Empty lines inside declarations are discouraged and there is a linter that enforces that they are not

present. This helps maintaining a uniform code style throughout all of mathlib.

You are however encouraged to add comments to your code: even a short sentence communicates *much* more than an empty line in the middle of a proof ever will!

Normal forms #

Some statements are equivalent. For instance, there are several equivalent ways to require that a subset `s` of a type is nonempty. For another example, given `a : α`, the corresponding element of `Option α` can be equivalently written as `Some a` or `(a : Option α)`. In general, we try to settle on one standard form, called the normal form, and use it both in statements and conclusions of theorems. In the above examples, this would be `s.Nonempty` (which gives access to dot notation) and `(a : Option α)`. Often, simp lemmas will be registered to convert the other equivalent forms to the normal form.

There is a special case to this rule. In types with a bottom element, it is equivalent to require `hlt : ⊥ < x` or `hne : x ≠ ⊥`, and it is not clear which one would be better as a normal form since both have their pros and cons. An analogous situation occurs with `hlt : x < ⊤` and `hne : x ≠ ⊤` in types with a top element. Since it is very easy to convert from `hlt` to `hne` (by using `hlt.ne` or `hlt.ne'` depending on the direction we want) while the other conversion is more lengthy, we use `hne` in *assumptions* of theorems (as this is the easier assumption to check), and `hlt` in *conclusions* of theorems (as this is the more powerful result to use). A common usage of this rule is with naturals, where `⊥ = 0`.

Use module doc delimiters `/-! -/` to provide section headers and separators since these get incorporated into the auto-generated docs, and use `/- -/` for more technical comments (e.g. TODOs and implementation notes) or for comments in proofs. Use `--` for short or in-line comments.

Documentation strings for declarations are delimited with `/-- -/`. When a documentation string for a declaration spans multiple lines, do not indent subsequent lines.

See our documentation requirements for more suggestions and examples.

Expressions in error or trace messages #

Inside all printed messages (such as, in linters, custom elaborators or other metaprogrammes), names and interpolated data should either be

- inline and surrounded by backticks (e.g., `m!"{foo}must have type{bar}"`), or
- on their own line and indented (via e.g. `indentD`)

The second style produces output like the following

```
Could not find model with corners for domain
  src
nor codomain
  tgt
of function
  f
```

Not all of mathlib may comply with this rule yet; that is a bug (and PRs fixing this are welcome).

Deprecation #

Deleting, renaming, or changing declarations can cause downstreams projects that rely on these definitions to fail to compile. Any publicly exposed theorems and definitions that are being removed should be gracefully transitioned by keeping the old declaration with a `@[deprecated]` attribute. This warns downstream projects about the change and gives them the opportunity to adjust before the declarations are deleted. Renamed definitions should use a deprecated `alias` to the new name.

Otherwise, when the deprecated definition does not have a direct replacement, the definition should be deprecated with a message, like so:

```
theorem new_name : ... := ...
@[deprecated (since := "YYYY-MM-DD")] alias old_name := new_name

@[deprecated "This theorem is deprecated in favor of using ... with ..." (since
:= "YYYY-MM-DD")]
theorem example_thm ...
```

The `@[deprecated]` attribute requires the deprecation date, and an alias to the new declaration or a string to explain how transition away from the old definition when a new version is no longer being provided.

The `deprecate to` command and `scripts/add_deprecations.sh` script can help generate alias definitions.

Deprecations for declarations with the `to_additive` attribute should ensure the deprecation is also properly tagged with `to_additive`, like so:

```
@[to_additive] theorem Group_bar {G} [Group G] {a : G} : a = a := rfl

-- Two deprecations required to include the `deprecated` tag on both the
additive
-- and multiplicative versions
@[deprecated (since := "YYYY-MM-DD")] alias AddGroup_foo := AddGroup_bar
@[to_additive existing, deprecated (since := "YYYY-MM-DD")] alias Group_foo :=
Group_bar
```

We allow, but discourage, contributors from simultaneously renaming declarations X to Y and W to X. In this case, no deprecation attribute is required for X, but it is for W.

Named instances do not require deprecations. Deprecated declarations can be deleted after 6 months.

Avoid `nonrec` #

The `nonrec` keyword tells Lean to assume that apparently recursive calls in the declaration body are not actually recursive, and instead look for declarations in other namespaces with the same name. Avoid `nonrec` when the recursive call conflicts with another declaration *in a namespace*, because then adding the namespace to that declaration is more informative (to both Lean and the user). If it conflicts with a declaration in the root namespace, then both `nonrec` and `_root_.[...]` are acceptable. Sometimes avoiding `nonrec` requires forgoing the use of dot notation within the body of that declaration. (There are currently many places in Mathlib that break this rule.)